

“192.168.1.1”, “192.168.1.2”, etc), calling `tcp_scan` for each host within the specified port range. It prints a message indicating the start of the scan on the network. Figure 2 depicts the enhanced functionality with `scanHost()` and `scanRange()`.

Defining function `tcp_scan()`

Loop through port range: It iterates through each port in the specified range.

Attempt connection: For each port, it attempts to establish a TCP connection with a timeout of 0.1 seconds.

Check connection status: If the connection attempt is successful (i.e., the port is open), it prints a message indicating an open TCP port.

Close socket: It closes the TCP socket after printing the message.

Exception handling: It includes exception handling to catch any errors during the connection attempt, passing over them without taking action. Figure 3 introduces the `tcp_scan()` function for comprehensive network analysis.

Defining function `network_scan()`

ARP packet creation: Creates an ARP packet with the target IP address (`target_ip`). Creates an Ethernet frame with the destination MAC address set to broadcast.

Sending and receiving packets: Sends the packet with a timeout of 3 seconds. Captures the response containing IP and MAC addresses.

Extracting information: Iterates over responses, extracting IP and MAC addresses into a list of dictionaries.

Printing results: Prints available devices in the network with their IP and MAC addresses in a formatted table. Figure 4 streamlines the network evaluation with `network_scan()` function.

Defining the `main()` function

Display options: It prints the available options: ‘1. TCP Port Scanner’ and ‘2. Network Scanner’.

Get user input: It prompts the user to enter their choice (either 1 or 2) using the `input()` function, and converts the input to an integer.

Handle user choice: If the user chooses option 1 (TCP Port Scanner), it prompts for an IP address, start port, and end port; it then calls the `scanHost()` function to perform a TCP port scan on the specified host within the specified port range.

If the user chooses option 2 (Network Scanner), it prompts for a target IP address (e.g., ‘172.17.3.1/24’) and calls the `network_scan()` function to perform a network scan.

If the user enters an invalid choice, it prints a message indicating so.

Execution: If this script is executed directly (i.e., `__name__ == “__main__”`), it calls the `main()` function to start the program. After the user selects an option and the scanning is complete, it prompts the user to press any key to close the program. Figure 5 defines the function `main()` coded with a menu driven model.

Result and output

Figure 6 shows the output of the TCP Port Scanner with range 1 to 1000.

Figure 7 depicts the Network Scanner capturing the available devices in the network.

In conclusion, the creation of a network and port scanner using Python represents a significant advancement in the realm of network security and management. Such tools are indispensable for network administrators and security professionals, providing them with the means to thoroughly analyse the state of their networks. The

```
24 def tcp_scan(ip, startPort, endPort):
25     for port in range(startPort, endPort + 1):
26         try:
27             tcp = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
28             tcp.settimeout(0.1)
29             if not tcp.connect_ex((ip, port)):
30                 with print_lock:
31                     print('[+] %s:%d/TCP Open' % (ip, port))
32                     tcp.close()
33         except Exception:
34             pass
```

Figure 3: Defining function `tcp_scan()`

```
36 def network_scan(target_ip):
37
38     arp = ARP(pdst=target_ip)
39
40     ether = Ether(dst="ff:ff:ff:ff:ff:ff")
41
42     packet = ether/arp
43
44     result = srp(packet, timeout=3, verbose=0)[0]
45
46     clients = []
47
48     for sent, received in result:
49
50         clients.append({'ip': received.psrc, 'mac': received.hsrc})
51
52
53     with print_lock:
54         print("Available devices in the network:")
55         print("IP" + " " * 18 + "MAC")
56         for client in clients:
57             print("{:16} {}".format(client['ip'], client['mac']))
```

Figure 4: Defining function `network_scan()`